

Diego Chuquillanqui Cuenca

During the third and fourth weeks of the Sensing the World module, I moved from learning about sensor fabrication to programming and integrating electronic components into embedded systems. We first learned a general approach to solving programming problems by identifying the objective, inputs, outputs, and constraints before planning the implementation. We used system diagrams and flowcharts to organize these ideas and analyzed the requirements for a security system that used sensors and an alarm. We also started working with GitHub and our engineering portfolios. In the next class, I connected an Arduino Nano ESP32 and began using MicroPython. I learned basic programming concepts such as variables, functions, modules, while loops, and how pins can be controlled through code. I then used these concepts in simple programs, including printing information and making an LED blink, and saved my work to my GitHub repository.

In the following class, I began combining programming concepts with physical circuits. I first built a circuit on a breadboard using a potentiometer to control an LED and saw how changing resistance affected its brightness. We then worked with switches and a photoresistor before applying these components to a prototype security system. The system used a potentiometer to arm or disarm it, a push button and photoresistor as inputs, and red and green LEDs to represent the alarm state. We also used AI to help generate the MicroPython code. I was able to see how a general prompt did not produce usable code because it did not include enough information about the board, pins, or intended behavior, but a much more specific prompt based on our system design produced code that worked. In the last class, I learned about PWM and used a switch to control a servo motor between two positions. We then tested a raindrop sensor under different levels of wetness, first measuring its voltage manually and then using the Arduino's ADC to automatically collect samples and save them to CSV files. Finally, we analyzed the results in a spreadsheet using averages, standard deviations, and a graph showing how the sensor voltage changed with wetness.

These activities helped me understand why planning a system before writing code is important. Creating a flowchart and identifying the inputs and outputs gave me a clearer picture of what the program actually needed to accomplish before implementing it. The security system also showed me how a microcontroller connects the software and hardware sides of an embedded system. It can read physical inputs from components such as a button or light sensor, process those values through a program, and then control an output such as an LED or motor. The AI assisted programming activity also showed me that the quality of generated code depends heavily on how clearly the problem is described. Giving the specific board, pin assignments, and expected behavior produced a much more useful result than simply asking for a program that reads sensors.

The raindrop sensor experiment in particular showed me how, when collecting sensor data, the way the experiment is performed affects how useful the results are. Our manual measurements used fewer samples than the automated measurements, and conditions such as how much of the sensor was wiped with water were difficult to reproduce exactly. This made the comparison less consistent. Using the same number of samples, applying controlled amounts of water, and returning the sensor to the same starting condition would make the experiment more

repeatable. Overall, these classes helped me see how system planning, circuits, programming, sensors, and data analysis work together. Instead of treating them as separate topics, I now better understand how they can be combined to build an embedded system that senses information from the physical world, processes it, and produces a useful response.